ECONOMIC INFORMATICS · BACHELOR · BUCHAREST UNIVERSITY OF ECONOMIC STUDIES

Multimedia

Lecture 2 — Raster and Vector Images

Prof. Adriana Dârdală (Reveiu) · Academic year 2026–2027

AGENDA

What we cover today

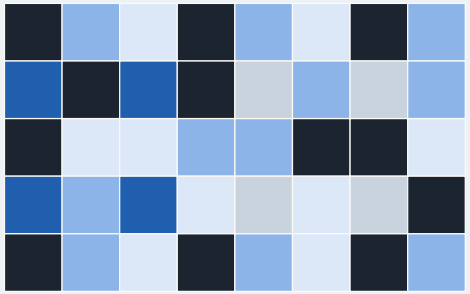
01	Light and colour: perception and colour models
02	The raster image: pixels, resolution, colour depth
03	Compression and raster file formats
04	Raster graphics in the browser: canvas
05	Vector graphics and SVG
06	Business use, lab preview and review

CONTEXT

Two ways to represent a picture

RASTER · BITMAP

A grid of pixels

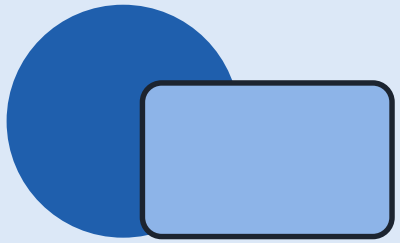


- Captured: camera, scanner
- Size depends on pixel count
- Pixelates when enlarged
- JPEG · PNG · WebP · AVIF

Best for: photographs, scanned documents, textures

VECTOR

A description of shapes



- Drawn: design tool, code
- Size depends on complexity
- Sharp at any size
- SVG · PDF · EPS · DXF

Best for: logos, icons, charts, maps, typography

01

Light and colour

From the spectrum to the numbers in a file

Light is a narrow band of the spectrum

ultraviolet ←

→ infrared



380 nm · violet

750 nm · red

Spectrum

A light source emits many wavelengths; the spectrum is the intensity at each one.

Three cone types

S, M and L cones respond to short, medium and long wavelengths: roughly blue, green, red.

Tristimulus

Three numbers describe a colour for a human. Hence three primaries on every screen.

Describing a colour: hue, saturation, brightness

Hue



Saturation



Lightness

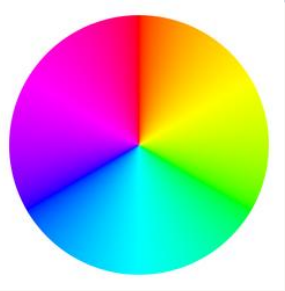


Relative perception: the same grey looks lighter on a dark background.



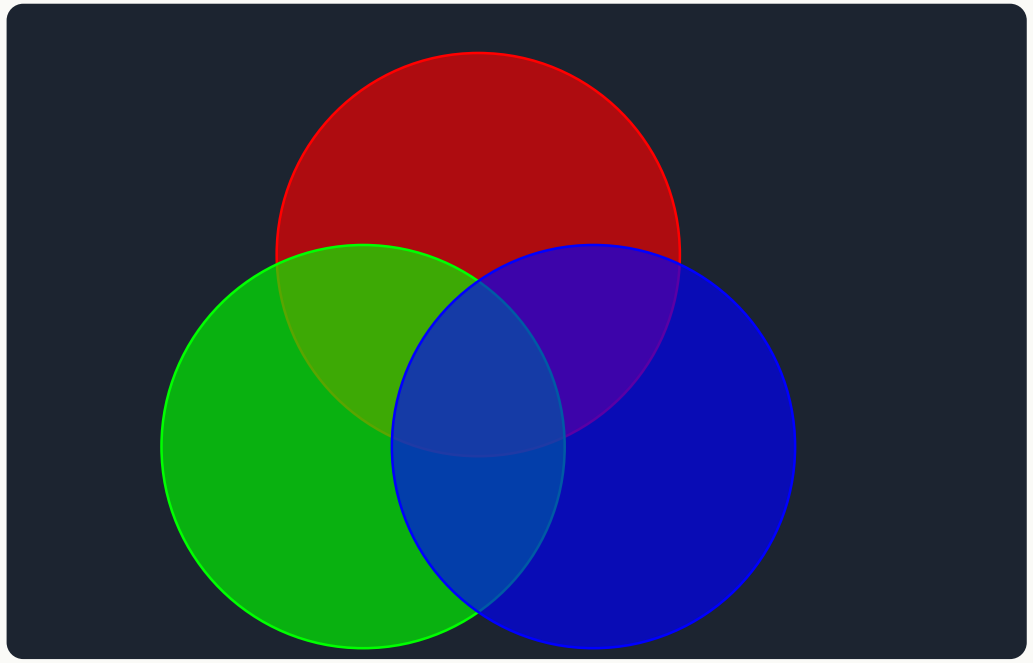


Term	Meaning
Hue	Colour family: angle 0–360° on the wheel
Saturation	Purity: 0% grey → 100% pure
Lightness	Light or dark, as perceived
Luminance	Measured light, weighted for the eye
Shade · tint · tone	+ black · + white · + grey



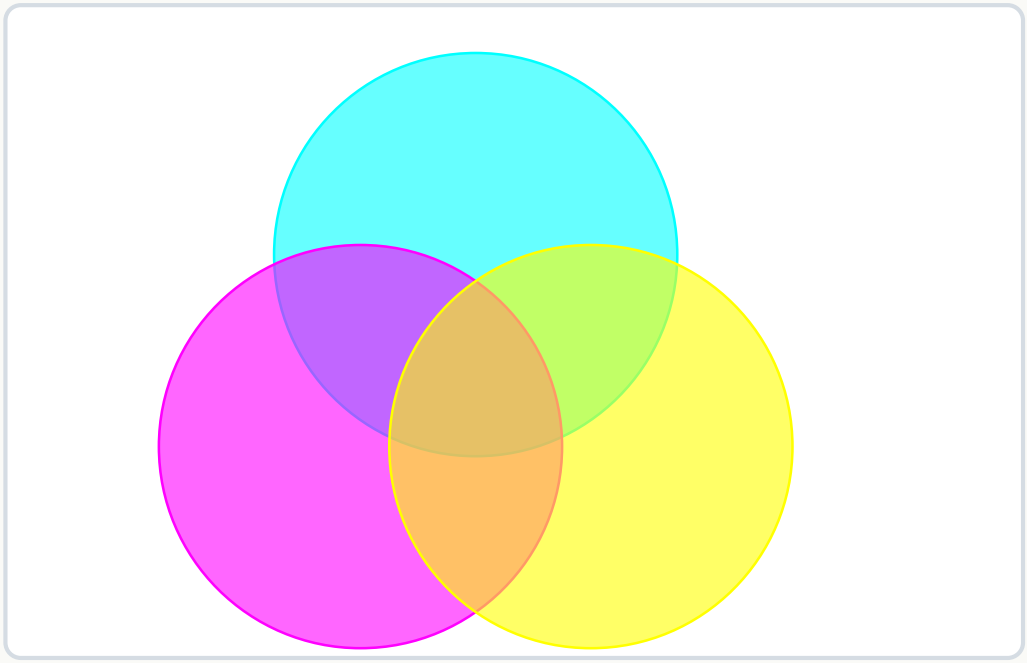
Additive and subtractive mixing

Additive · RGB · light



R + G + B = white

Subtractive · CMY(K) · ink



C + M + Y ≈ black

Complementary pairs: **red–cyan · green–magenta · blue–yellow**

Colour models, colour spaces and gamut

ABSTRACT

Colour model

The method: which numbers describe a colour. RGB, CMYK, HSL, Lab.

CONCRETE

Colour space

A model with exact primaries and white point. sRGB, Display P3, Adobe RGB.

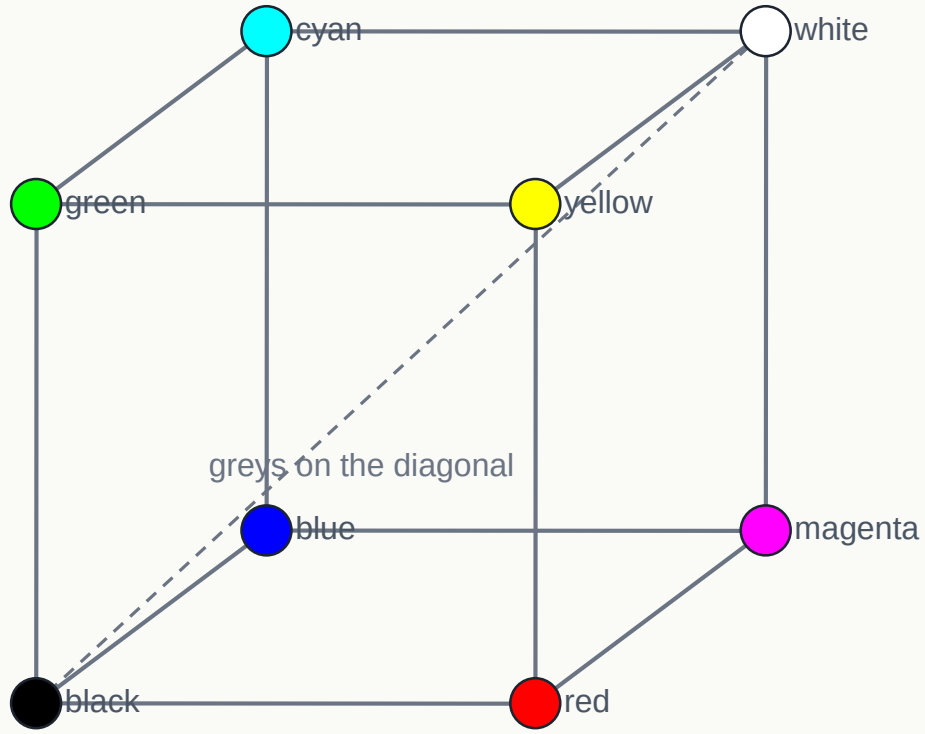
RANGE

Gamut

All colours a space or device can reproduce. A monitor ≠ a printer.

Colour space	Gamut	Where it is used
sRGB	Reference (≈ 35% of visible colours)	Web default, most images
Display P3	≈ 25% larger than sRGB	Phones, laptops, CSS `color(display-p3 …)`
Adobe RGB	Larger, especially greens	Professional photography, print
Rec. 2020	Very large	HDR video, UHD TV

The RGB model



256^3 = 16,777,216 colours

Notation	Same colour
Hexadecimal	`#1F5FAD`
Short hex	`#F00` = `#FF0000`
Decimal	`rgb(31 95 173)`
With alpha	`rgb(31 95 173 / 50%)`

1F → 31 · 5F → 95 · AD → 173

CMYK: colour for print

```
// naive RGB → CMY, values in [0, 1]
C = 1 − R     M = 1 − G     Y = 1 − B

// with black (K)
K = 1 − max(R, G, B)
C = (1 − R − K) / (1 − K)   // same for M, Y
```

Example: `#1F5FAD` = (0.12, 0.37, 0.68)

K = 1 − 0.68 = 0.32 → **C 82% · M 45% · Y 0% · K 32%**

Why add black (K)?

- Black ink is the cheapest
- Deep, neutral blacks instead of brown
- Less ink: dries faster
- Sharp, legible text

In practice

- Use ICC profiles for the paper and press
- Check out-of-gamut colours before printing

HSV and HSL: models for people



	HSV / HSB	HSL
Hue	0–360°	0–360°
Saturation	0–100%	0–100%
3rd value	Value: 100% = pure hue	Lightness: 50% = pure hue
Shape	Cone (hexcone)	Double cone
Used in	Photoshop, GIMP pickers	CSS `hsl()`

A palette from one hue:

20%	35%	50%	65%	80%	92%

Luma and chroma: YIQ and YCbCr

```
Y  =  0.299 R + 0.587 G + 0.114 B   // luma
Cb =  B − Y   (scaled + offset)        // blue diff.
Cr =  R − Y   (scaled + offset)        // red diff.

// YIQ (NTSC):  I ≈ R − Y,  Q ≈ B − Y
```



Brightness carries the detail; colour can be stored at lower resolution.

Why separate them?

The eye resolves brightness detail far better than colour detail.

Where it is used

- JPEG, WebP, AVIF
- MPEG, H.264, VP9, AV1 video
- Grayscale conversion (Lab 4)

Perceptual models: CIE Lab and OKLCH

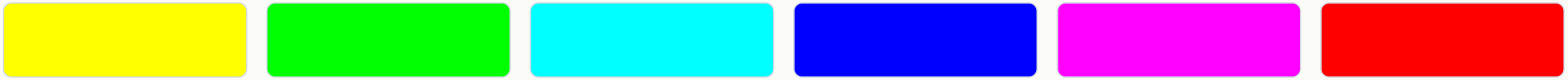
CIE Lab

- **L** lightness 0–100 · **a** green ↔ red · **b** blue ↔ yellow
- Device-independent: describes appearance, not production

OKLCH (CSS Color 4)

- **L** lightness · **C** chroma · **H** hue angle
- Perceptually uniform; reaches Display P3 colours

Same HSL lightness (50%) — very different perceived brightness:



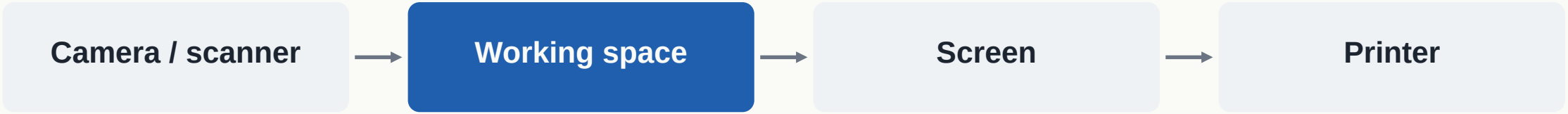
Same OKLCH lightness (70%) — even brightness:



Comparing the colour models

Model	Type	Components	Typical use
RGB	Additive, device-dependent	Red, green, blue	Screens, cameras, web images
CMYK	Subtractive, device-dependent	Cyan, magenta, yellow, black	Printing
HSV / HSL	Cylindrical RGB	Hue, saturation, value / lightness	Colour pickers, CSS `hsl()`
YIQ / YCbCr	Luma + chroma	Luma, two colour differences	TV, JPEG, video codecs
Grayscale	One channel	Intensity 0–255 (8 bits)	Masks, scans, processing
CIE Lab	Perceptual, device-independent	Lightness, green–red, blue–yellow	Colour management, conversion
OKLCH	Perceptual, device-independent	Lightness, chroma, hue	Modern CSS palettes, P3 colours

Colour management and colour in CSS



Each arrow is a conversion through **ICC profiles**; out-of-gamut colours are mapped or lost.

```
.brand  { color: #1f5fad; }                /* sRGB */
.accent { color: hsl(213 70% 40%); }       /* sRGB */
.vivid  { color: oklch(62% 0.21 255); }    /* perceptual */
.p3     { color: color(display-p3 0 0.4 1); }
.soft   { color: color-mix(in oklch,
                  var(--brand) 40%, white); }
```

Accessibility

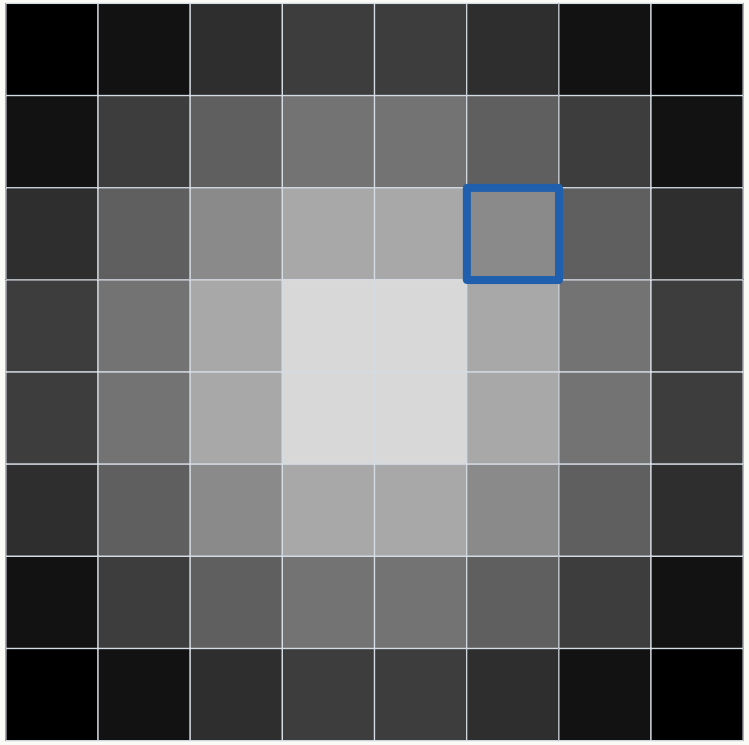
- Text contrast at least **4.5:1** (WCAG 2.2 AA), 3:1 for large text
- Never rely on colour alone: add icons or labels
- Prefer blue/orange to red/green pairs

02

The raster image

Pixels, resolution and colour depth

An image is a function sampled on a grid



highlighted: pixel (x = 5, y = 2) = 138

```
grayscale:  f(x, y) → intensity
colour:     f(x, y) → ( r(x, y), g(x, y), b(x, y) )
```

Sampling

Space is divided into a grid: width × height pixels.

Quantisation

Each pixel's colour is rounded to a finite set of values: the colour depth.

Three factors decide quality

WIDTH × HEIGHT

Pixel dimensions

How much detail exists. 4000 × 3000 = 12 megapixels.

BITS PER PIXEL

Colour depth

How many colours a pixel can take. 24 bits → 16.7 million.

PPI / DPI

Resolution

Pixels per inch when shown or printed. Print: 300 ppi.

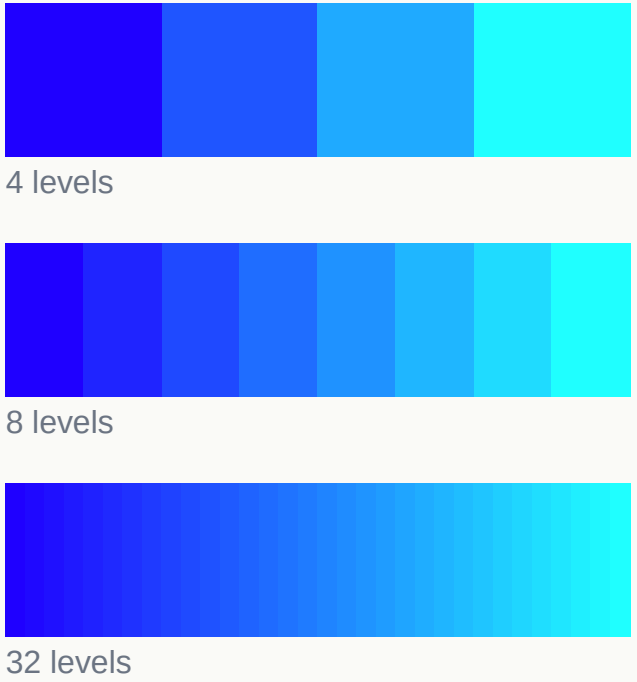
How many pixels for a 10 × 15 cm print at 300 ppi?

10 cm = 3.94 in → 3.94 × 300 ≈ 1181 px · 15 cm = 5.91 in → 5.91 × 300 ≈ 1772 px → **1181 × 1772 px (2.1 MP)**

Colour depth

Bits per pixel	Colours	Name	Typical use
1	2	Monochrome	Fax, line-art scans
8	256 grey levels	Grayscale	Masks, medical, documents
8	256 from a palette	Indexed colour	GIF, PNG-8, icons
16	65,536	High colour (5-6-5)	Old displays, embedded
24	16,777,216	True colour (8 × 3)	Photos, the web
32	16.7 M + 256 alpha levels	True colour + alpha	PNG, WebP with transparency
30–48	Over 1 billion	Deep colour / HDR	RAW photos, HDR, pro video

Banding with too few levels



4 levels

8 levels

32 levels

Indexed colour, palettes and dithering

Palette

0 → #1C2430

1 → #1F5FAD

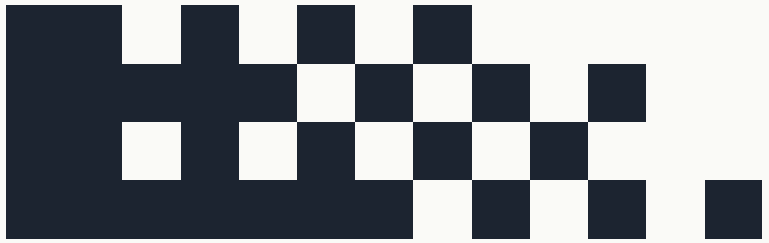
2 → #8DB4E8

3 → #FAFAF7

Pixels store indices

0	1	1	2	3
1	1	2	3	3
1	2	2	3	3
2	2	3	3	3

Dithering: two colours, many apparent tones



dark → light with only black and white pixels

Good for

Logos, icons, flat art: few colours, 1 byte per pixel

Bad for

Photos: banding, or noise if dithered

Formats

GIF, PNG-8, BMP 8-bit

Calculating the size of an uncompressed image

size (bytes) = width × height × bits per pixel ÷ 8

Image	Calculation	Raw size
640 × 480, 8-bit indexed	640 · 480 · 8 / 8	307,200 B ≈ 0.31 MB
320 × 240, 16-bit high colour	320 · 240 · 16 / 8	153,600 B ≈ 0.15 MB
1920 × 1080, 24-bit	1920 · 1080 · 24 / 8	6,220,800 B ≈ 6.2 MB
4000 × 3000 (12 MP), 24-bit	4000 · 3000 · 3	36,000,000 B = 36 MB
1000 × 1000 PNG with alpha, 32-bit	1000 · 1000 · 4	4,000,000 B = 4 MB

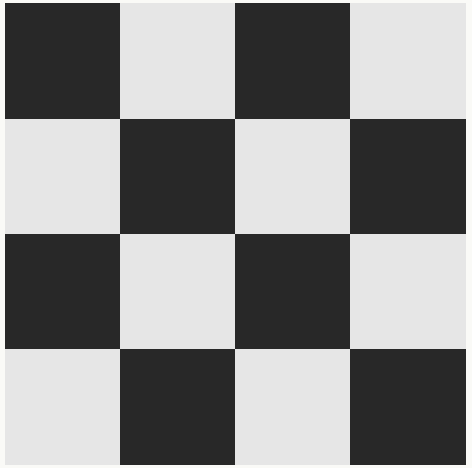
02 · RASTER

36 MB → 3 MB

is what JPEG typically does to a 12-megapixel photo.

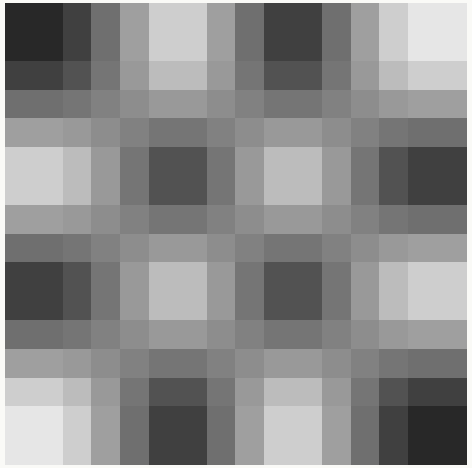
4000 × 3000 pixels × 3 bytes, compressed about 12:1 at good quality. The difference is invisible to most viewers.

Scaling and interpolation



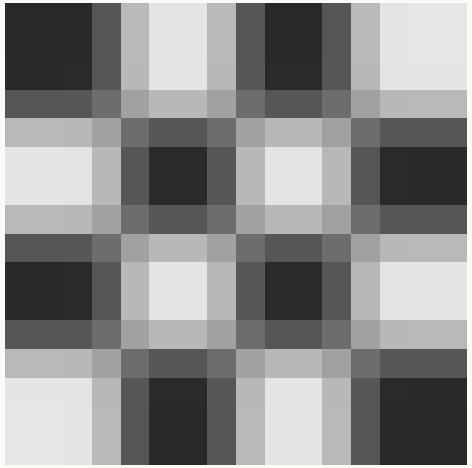
Nearest neighbour

Blocks; keeps hard edges. Pixel art.



Bilinear

4 neighbours. Smooth, a bit soft.

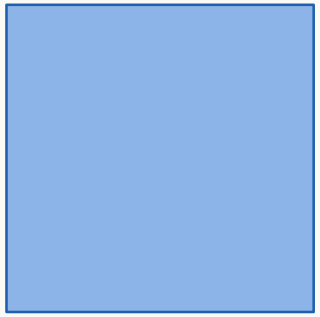


Bicubic / Lanczos

16+ neighbours. Sharp. Editors' default.

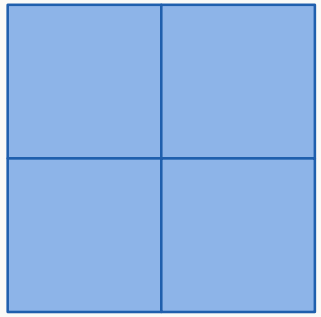
```
img.pixel-art { image-rendering: pixelated; }   /* nearest neighbour in CSS */
```

Screens: CSS pixels and device pixels



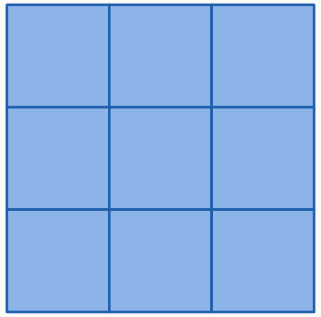
devicePixelRatio = 1

1 device pixel



devicePixelRatio = 2

4 device pixels



devicePixelRatio = 3

9 device pixels

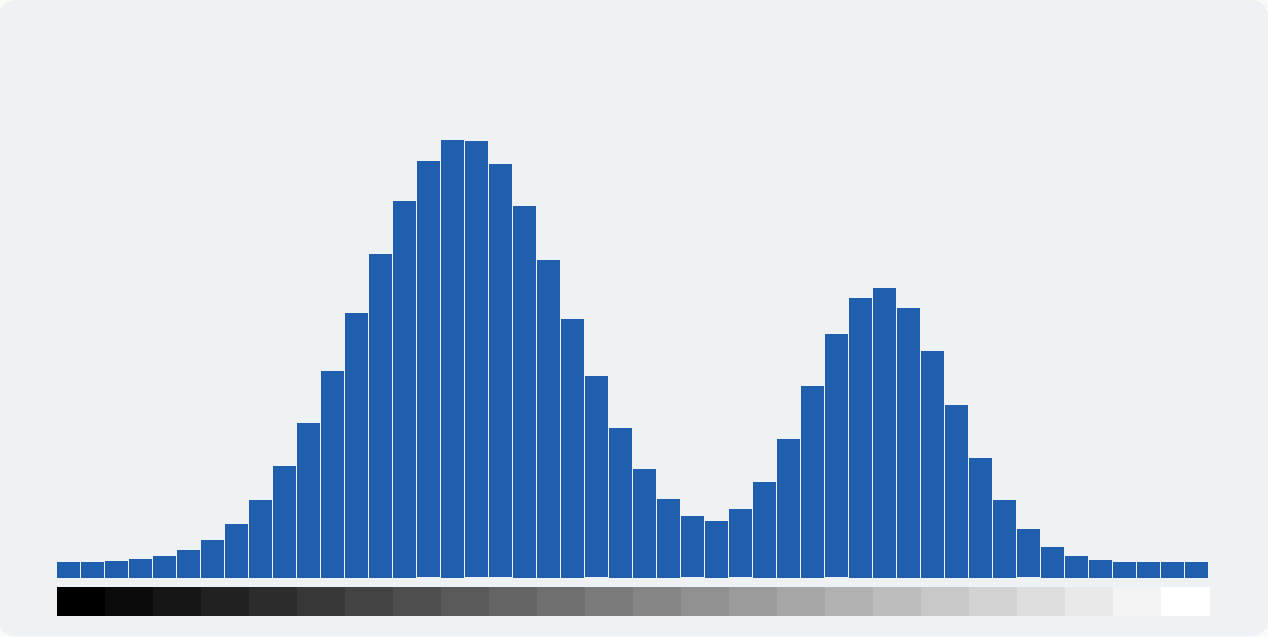
```
<!-- fixed size: density -->
<img src="logo.png"
  srcset="logo.png 1x, logo@2x.png 2x"
  width="200" height="60" alt="Shop">

<!-- responsive: width + sizes -->
<img src="p-800.jpg" alt="Shoe"
  srcset="p-400.jpg 400w, p-800.jpg 800w,
          p-1600.jpg 1600w"
  sizes="(max-width: 600px) 100vw, 50vw">
```

Rule: image pixels needed = CSS display size × devicePixelRatio. Serve that, not more.

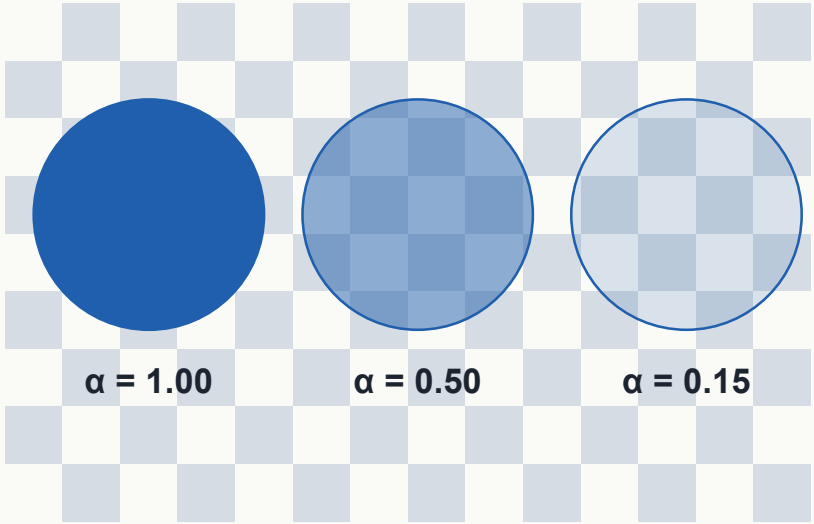
`1x`/`2x`: fixed-size images (logos, icons) · `w` + `sizes`: images whose width depends on the layout

Brightness, contrast and the histogram



Histogram shape	Diagnosis
Crowded on the left	Underexposed, too dark
Crowded on the right	Overexposed, too light
Narrow, in the middle	Low contrast, flat
Spike at 0 or 255	Clipping: detail lost
Spread across the range	Good tonal range

Transparency and the alpha channel




```
result = α·F + (1 − α)·B
// per channel R, G, B
```

Format	Transparency
JPEG	None
GIF	1-bit: on / off, jagged edges
PNG	8-bit alpha
WebP, AVIF	8-bit alpha (lossy or lossless)
SVG	Opacity per shape, fill, stroke

03

Compression and raster file formats

Removing redundancy and invisible detail

Lossless and lossy compression

EXACT

Lossless

- Decompressed = original, bit for bit
- Removes statistical redundancy
- Ratios around 2:1 to 3:1 for photos
- RLE, Huffman, LZW, Deflate · PNG, GIF

APPROXIMATE

Lossy

- Decompressed ≈ original
- Also drops what we barely perceive
- Ratios 10:1 to 100:1 and more
- DCT, quantisation · JPEG, WebP, AVIF

compression ratio = B_0 / B_1

Asymmetric: slow encoding, fast decoding

B_0 = bits before · B_1 = bits after. Example: 36 MB → 3 MB gives 12:1.

Three kinds of redundancy in images

01

Spatial

Neighbouring pixels are similar

Exploited by: Run lengths, prediction, DCT

LOSSLESS AND LOSSY

02

Coding

Some values are more frequent

Exploited by: Huffman, arithmetic coding

LOSSLESS

03

Psychovisual

The eye ignores fine colour and texture

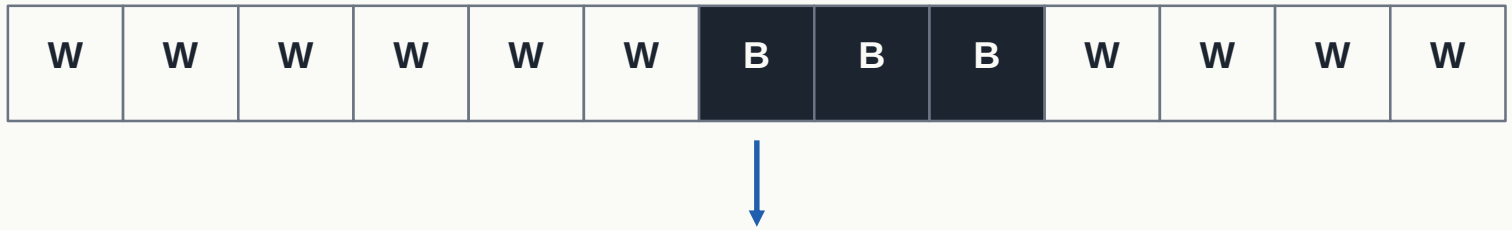
Exploited by: Chroma subsampling, quantisation

LOSSY ONLY

Video adds **temporal** redundancy between frames (Lecture 9).

Run-length encoding (RLE)

Input: 13 pixels



Output: 3 runs

6 × W | **3 × B** | **4 × W**

13 symbols → 6 symbols

Worst case `ABCD` → `1A1B1C1D`: twice the size.

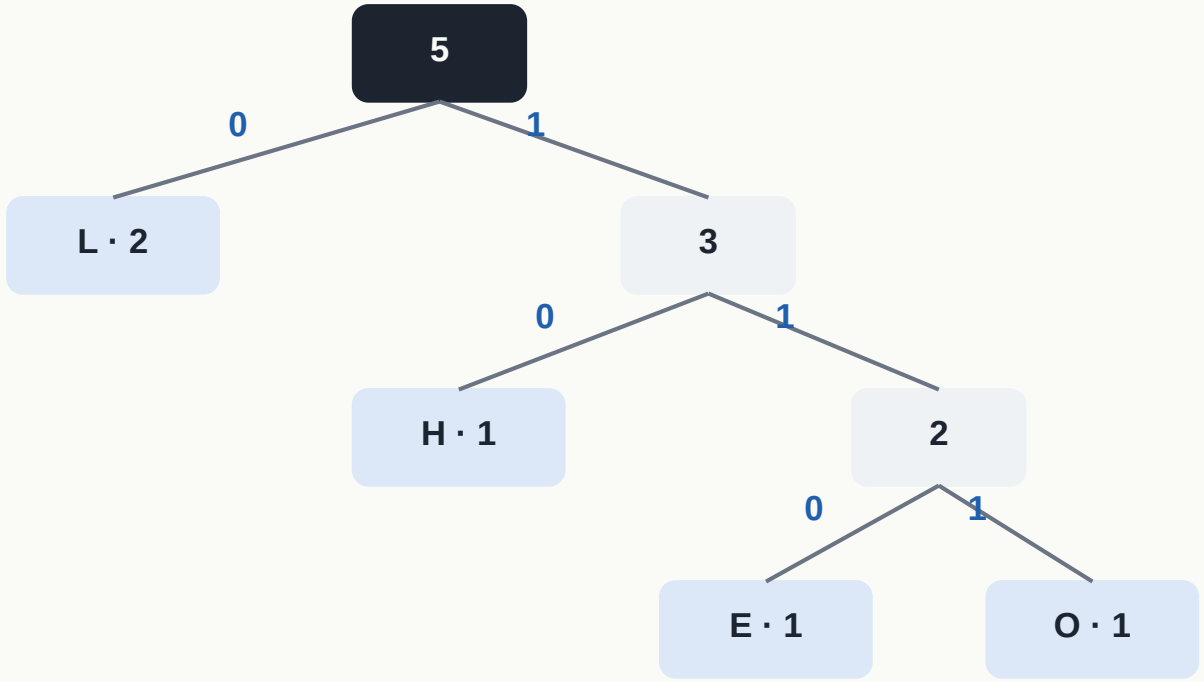
Works for

- Flat graphics, masks
- Scans, fax
- Zeros after JPEG quantisation

Fails for

- Photos: few identical neighbours
- Can make files larger

Variable-length coding: Huffman



Symbol	Count	Code	Bits
L	2	`0`	2 × 1 = 2
H	1	`10`	2
E	1	`110`	3
O	1	`111`	3
Total			**10 bits**

ASCII: 5 × 8 = **40 bits** → Huffman: **10 bits**

Prefix-free: `0 10 110 0 111` decodes without separators

Dictionary coding: LZW

Input: `ABABBABCABABBA` · initial dictionary: 1 = A, 2 = B, 3 = C

Output	**1**	**2**	**4**	**5**	**2**	**3**	**4**	**6**	**1**
String	A	B	AB	BA	B	C	AB	ABB	A
New entry	4 = AB	5 = BA	6 = ABB	7 = BAB	8 = BC	9 = CA	10 = ABA	11 = ABBA	—

Result

14 symbols → 9 codes. Ratio 14 / 9 ≈ 1.56.

Key idea

The dictionary is rebuilt by the decoder; it is never sent.

Used in

GIF, TIFF (LZW) · PNG, ZIP, gzip (Deflate = LZ77 + Huffman)

Arithmetic coding

Symbol	Prob.	Range
A	0.20	[0, 0.2)
B	0.10	[0.2, 0.3)
C	0.20	[0.3, 0.5)
D	0.05	[0.5, 0.55)
E	0.30	[0.55, 0.85)
F	0.05	[0.85, 0.9)
$	0.10	[0.9, 1.0)

Encode CAEE$	Low	High	Range
start	0	1.0	1.0
C	0.3	0.5	0.2
A	0.30	0.34	0.04
E	0.322	0.334	0.012
E	0.3286	0.3322	0.0036
$	0.33184	0.33220	0.00036

Any number in [0.33184, 0.33220) encodes the whole message.

JPEG: a six-step pipeline

1	2	3	4	5	6
Colour transform	**Downsampling**	**Block splitting**	**DCT**	**Quantisation**	**Entropy coding**
RGB → YCbCr	Chroma 4:2:0	8 × 8 blocks	Pixels → frequencies	Divide and round	Zigzag, RLE, Huffman

Blue steps lose information · the others are reversible

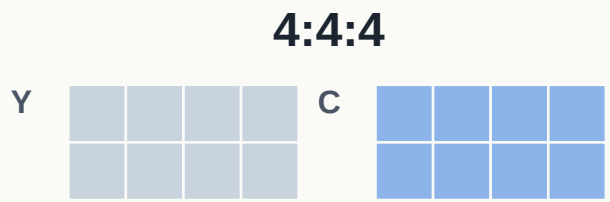
Good for

Photographs, natural scenes, gradual colour changes

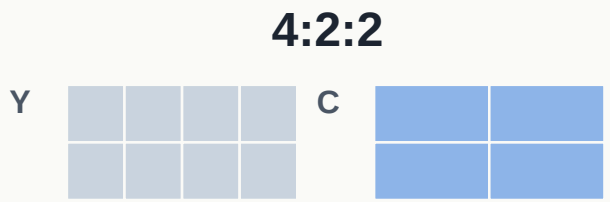
Bad for

Text, line art, logos, screenshots: artefacts at sharp edges

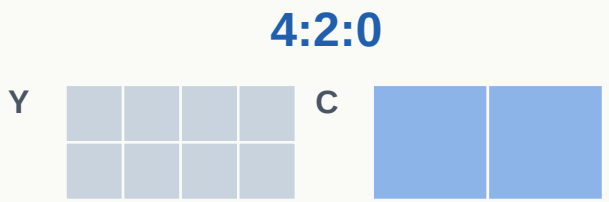
Steps 1–3: colour transform, subsampling, blocks



3 samples / px



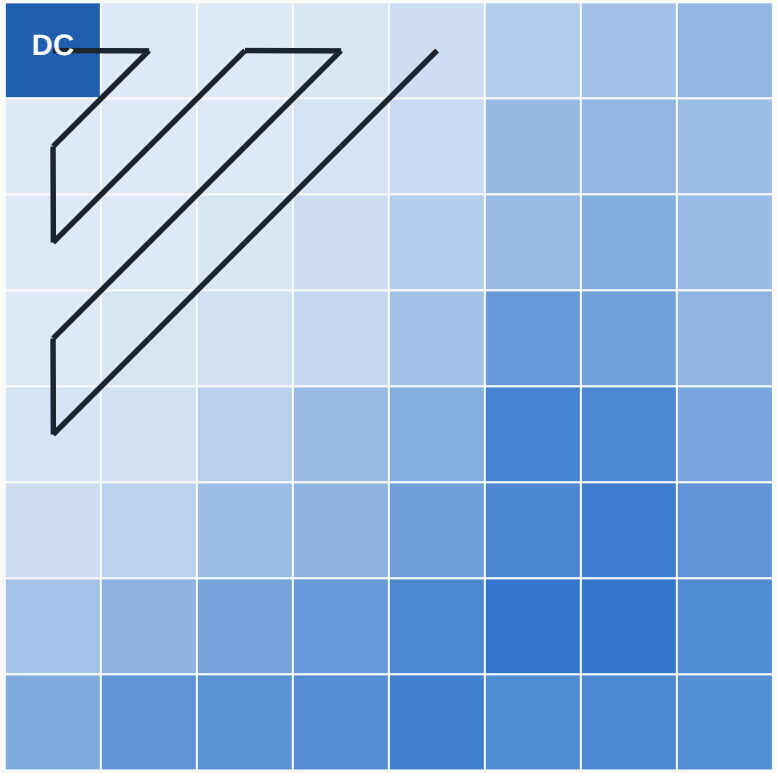
2 samples / px



1.5 samples / px

Step	What happens	Why
1 · RGB → YCbCr	Luma Y and chroma Cb, Cr in separate channels	Brightness gets the most bits
2 · Subsampling	Chroma stored at ½ or ¼ resolution	Eye resolves colour poorly: 50% less data
3 · Blocks	Each channel cut into 8 × 8 pixel blocks	DCT works on small, similar areas

Steps 4–6: DCT, quantisation and entropy coding



low frequency ↖ ↘ high frequency (fine detail)

Step	Operation	Effect
4 · DCT	64 pixels → 64 frequency coefficients	Energy concentrated top-left
5 · Quantisation	Divide by table, round to integer	**Most high frequencies → 0 (loss)**
6 · Zigzag	Read low → high frequency	Zeros grouped at the end
6 · DPCM + RLE	DC as difference; AC as (zeros, value)	Short sequences
6 · Huffman	Variable-length codes	Final bit stream

Quality, artefacts and JPEG modes

Quality setting

- Scales the quantisation tables
- Web: **70–85** usually enough
- Not comparable across encoders

Artefacts

- **Blocking:** visible 8 × 8 squares
- **Ringing:** halos at edges
- **Bleeding:** colour smears

Generation loss

- Each re-save loses more
- Edit a lossless master
- Export to JPEG once

Mode	How it works	Use
Sequential (baseline)	One scan, top to bottom	Default
Progressive	Coarse full image first, then refinements	Web: perceived speed
Hierarchical	Several resolutions, each refining the previous	Rare
Lossless	Prediction instead of DCT	Rare: PNG is used instead

PNG and GIF: lossless formats

LEGACY

GIF · 1987

- 256 colours from a palette, LZW
- 1-bit transparency, jagged edges
- Animation: frames + delays
- Today: replace animations with MP4/WebM video

LOSSLESS STANDARD

PNG · 1996

- Lossless: filter rows, then Deflate
- PNG-8 (palette), PNG-24, PNG-32 (alpha)
- Full 8-bit alpha, colour profiles
- Screenshots, diagrams, UI, cut-outs

```
PNG filter "Sub":  stored = pixel − left neighbour   // smooth rows → small numbers
```

WebP, AVIF and JPEG XL: modern formats

Format	Year · origin	Based on	Typical saving vs JPEG	Browser support (2026)
WebP	2010 · Google	VP8 intra-frame	25–35%	All browsers
AVIF	2019 · AOMedia	AV1 intra-frame, HEIF	30–50%	All major browsers since 2024
JPEG XL	2022 · JPEG committee	New codec; JPEG recompression	Similar to AVIF	Safari 17+; Chrome behind a flag
HEIC	2015 · MPEG	HEVC intra-frame	≈ 50%	Safari only: convert on upload

Always provide a fallback with `<picture>`: modern format first, JPEG or PNG last. Check support on caniuse.com.

Raster formats compared

Format	Compression	Colours	Alpha	Animation	Best for
JPEG	Lossy (DCT)	24-bit	No	No	Photos, broad compatibility
PNG	Lossless (Deflate)	8 / 24 / 48-bit	Yes	APNG	Screenshots, UI, cut-outs
GIF	Lossless (LZW)	256 palette	1-bit	Yes	Legacy; avoid
WebP	Lossy or lossless	24-bit	Yes	Yes	Web photos and graphics
AVIF	Lossy or lossless (AV1)	Up to 12-bit/ch	Yes	Yes	Web photos, HDR
SVG	Text (gzip on server)	Any	Yes	Yes	Logos, icons, charts
BMP	None or RLE	1–32-bit	Limited	No	Raw pixels; not for web
TIFF	None, LZW, ZIP, JPEG	Up to 32-bit/ch, CMYK	Yes	No	Scanning, print, archives
RAW	Lossless or none	12–14-bit sensor	—	No	Photo masters

Choosing an image format

Can it be drawn as shapes?	**SVG**	Logos, icons, charts, maps
Is it a photo or continuous tone?	**AVIF / WebP + JPEG fallback**	Product photos, banners
Sharp edges, text, transparency?	**PNG or lossless WebP**	Screenshots, UI, cut-outs
Does it move?	**MP4 / WebM video**	Instead of animated GIF

Then: size for the slot (CSS size × devicePixelRatio) → compress (Squoosh, sharp) → serve with `<picture>` + `srcset`.

Serving images on the web

```
<picture>
  <source type="image/avif"
    srcset="shoe-640.avif 640w, shoe-1280.avif 1280w"
    sizes="(max-width: 700px) 100vw, 640px">
  <source type="image/webp"
    srcset="shoe-640.webp 640w, shoe-1280.webp 1280w"
    sizes="(max-width: 700px) 100vw, 640px">
  <img src="shoe-640.jpg"
       alt="Red running shoe, side view"
       width="640" height="480"
       fetchpriority="high" decoding="async">
</picture>
```

Attribute	Effect
`type`	Skip unsupported formats
`srcset` + `sizes`	Pick the right resolution
`width`, `height`	No layout shift
`loading="lazy"`	Defer off-screen images
`fetchpriority`	Load the hero image first

Measured by **Core Web Vitals:** LCP (load) and CLS (layout shift).

04

Raster graphics in the browser: canvas

A bitmap you draw on with JavaScript

The canvas element

```
<canvas id="stage" width="600" height="400">
  Fallback text for old browsers
</canvas>

const canvas = document.querySelector("#stage");
const ctx = canvas.getContext("2d");

// sharp on high-density screens
const dpr = window.devicePixelRatio || 1;
canvas.width  = 600 * dpr;
canvas.height = 400 * dpr;
canvas.style.width  = "600px";
canvas.style.height = "400px";
ctx.scale(dpr, dpr);
```

Immediate mode

Draws pixels and forgets the shapes

Context

`getContext('2d')` · `'webgpu'` · `'webgl2'`

Bitmap size ≠ CSS size

Attributes set pixels; CSS only stretches

Drawing shapes, paths and text

```
ctx.fillStyle = "#1f5fad";
ctx.fillRect(20, 20, 160, 100);          // direct

ctx.beginPath();                           // path
ctx.moveTo(220, 120);
ctx.lineTo(300, 20);
ctx.lineTo(380, 120);
ctx.closePath();
ctx.lineWidth = 4;
ctx.strokeStyle = "#1c2430";
ctx.stroke();

ctx.beginPath();
ctx.arc(480, 70, 50, 0, 2 * Math.PI);    // circle
ctx.fill();

ctx.font = "bold 24px Arial";
ctx.fillText("Sale -20%", 20, 180);
```

Group	Methods / properties
Rectangles	`fillRect` `strokeRect` `clearRect`
Paths	`beginPath` `moveTo` `lineTo` `arc` `closePath`
Curves	`quadraticCurveTo` `bezierCurveTo`
Draw	`fill()` `stroke()`
Style	`fillStyle` `lineWidth` `lineCap` `font`
State	`save()` `restore()`

Images and pixels on a canvas

```
ctx.drawImage(img, x, y);                          // as is
ctx.drawImage(img, x, y, w, h);                    // scaled
ctx.drawImage(img, sx, sy, sw, sh,                 // crop
                   dx, dy, dw, dh);
```

```
const img = ctx.getImageData(0, 0, W, H);
const d = img.data;                 // [R,G,B,A, R,G,B,A, …]
const i = (y * W + x) * 4;          // pixel (x, y)
d[i] = 255 - d[i];                  // invert red
ctx.putImageData(img, 0, 0);
```

ImageData.data for 4 pixels

	R	G	B	A
	255	0	0	255
	0	0	0	255
	255	255	255	255
	203	53	148	255

Cross-origin image without CORS → canvas **tainted** → `getImageData` throws.

Preview: point operations and convolution

Point filter	Formula per pixel
Grayscale	`v = 0.299 r + 0.587 g + 0.114 b`
Negative	`r' = 255 − r` (same for g, b)
Brightness	`r' = r + k`, clamped to 0–255
Threshold	`v = luma ≥ t ? 255 : 0`
Sepia	`r' = 0.393 r + 0.769 g + 0.189 b` …

Blur (box)

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Sharpen

0	−1	0
−1	5	−1
0	−1	0

Edges

−1	−1	−1
−1	8	−1
−1	−1	−1

Next week · Lecture 4

Histogram, convolution, Web Workers and OffscreenCanvas: the image editor of Lab 4.

05

Vector graphics and SVG

Shapes described by geometry, drawn at any size

What a vector image stores

Advantages

- Sharp at every size: scale without loss
- Small files for logos, icons, charts
- Editable: each shape remains an object
- Text stays text: searchable, accessible

Limitations

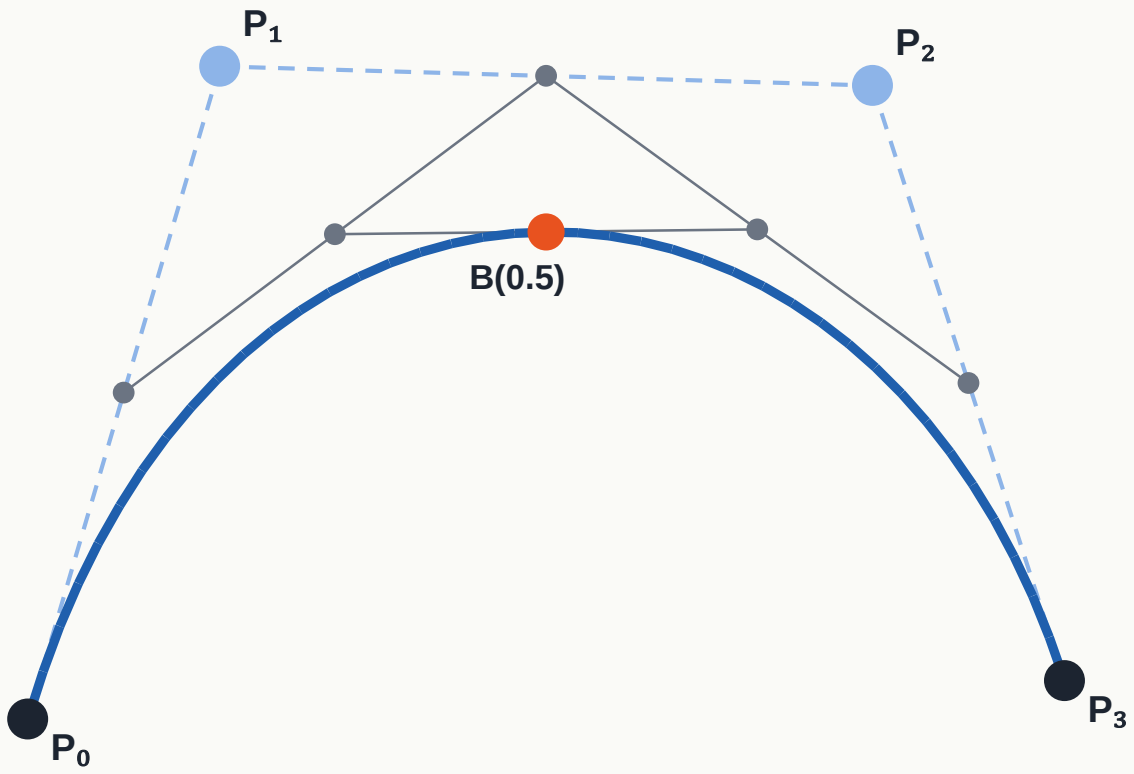
- Unsuitable for photos and fine textures
- Complex art can render slowly
- Needs rasterisation to be displayed
- Look depends on the renderer

Format	Origin	Typical use
SVG	W3C, XML	Web graphics, icons, charts, animation
PDF / EPS / AI	Adobe, PostScript	Print, illustration, logo masters
DXF / DWG	Autodesk	CAD drawings, architecture
SHP / GeoJSON	Esri / IETF	Maps and GIS data

Raster versus vector

	Raster	Vector
Stores	A grid of coloured pixels	Shapes, curves, text and styles
Origin	Captured: camera, scanner	Drawn: design tool, code, data
File size depends on	Pixel count × colour depth	Number and complexity of shapes
Enlarging	Pixelation, blur	Always sharp
Editing	Per pixel: paint, filters	Per object: move, recolour, reshape
Best for	Photos, textures, scans	Logos, icons, type, charts, maps
Web formats	JPEG, PNG, WebP, AVIF	SVG
Conversion	Tracing: approximate	Rasterisation: exact

Bézier curves




```
quadratic:
  B(t) = (1−t)²P₀ + 2(1−t)t·P₁ + t²P₂
cubic:
  B(t) = (1−t)³P₀ + 3(1−t)²t·P₁
       + 3(1−t)t²·P₂ + t³P₃      t ∈ [0, 1]
```

Where you meet them

- SVG paths `Q`, `C` · canvas `bezierCurveTo`
- Pen tool in Figma, Illustrator
- Fonts: TrueType (quadratic), CFF (cubic)
- CSS easing `cubic-bezier()`

2D transformations

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \qquad \begin{aligned} x' &= a \cdot x + c \cdot y + e \\ y' &= b \cdot x + d \cdot y + f \end{aligned}$$

```
// rotate a 100×60 box around its centre
ctx.save();
ctx.translate(x + 50, y + 30);
ctx.rotate(Math.PI / 6);          // 30°
ctx.fillRect(-50, -30, 100, 60);
ctx.restore();
```

Transform	Matrix (a, b, c, d, e, f)	Canvas	SVG / CSS
Translate	(1, 0, 0, 1, tx, ty)	`translate(tx, ty)`	`translate(tx ty)`
Scale	(sx, 0, 0, sy, 0, 0)	`scale(sx, sy)`	`scale(sx sy)`
Rotate θ	(cos, sin, −sin, cos, 0, 0)	`rotate(rad)`	`rotate(deg)`

SVG: Scalable Vector Graphics

```
<svg xmlns="http://www.w3.org/2000/svg"
     viewBox="0 0 200 120" width="400">
  <title>Shop logo</title>
  <rect x="10" y="10" width="180" height="100"
        rx="16" fill="#1f5fad"/>
  <circle cx="60" cy="60" r="28" fill="#8db4e8"/>
  <text x="100" y="68" fill="#fafaf7"
        font-family="Arial" font-size="24">Shop</text>
</svg>
```



rendered at any size, always sharp

Text

XML, gzip, Git

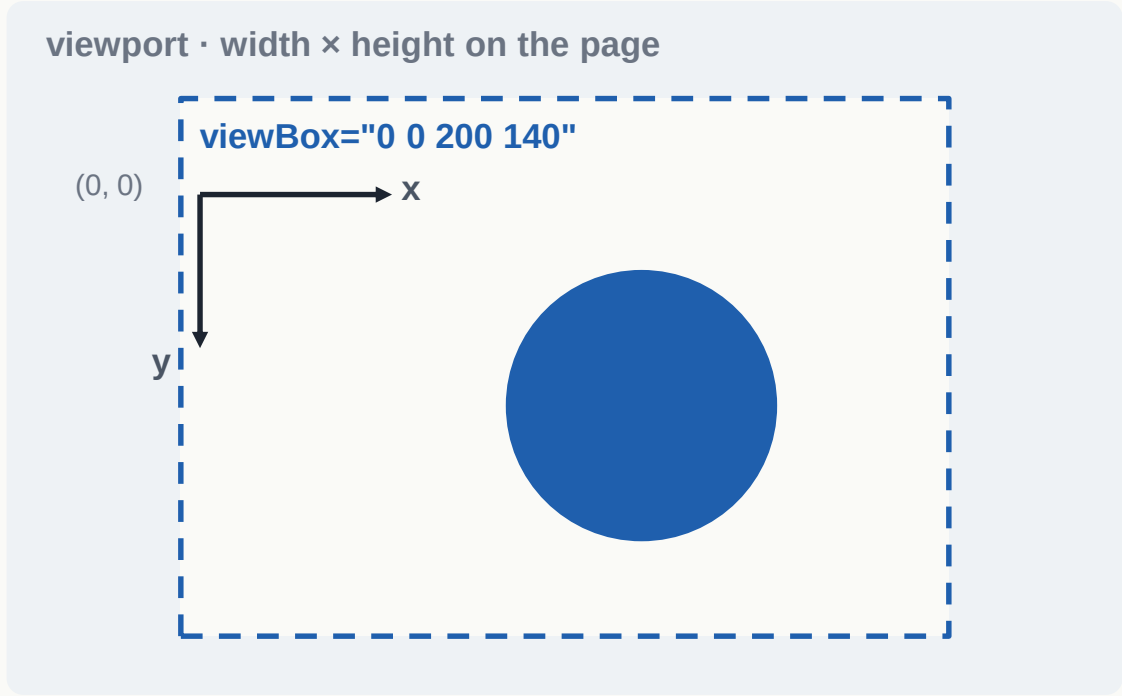
DOM

CSS, JS, events

Mixed

Shapes, text, images

The coordinate system and viewBox




```
svg { width: 100%; height: auto; }   /* responsive */
```

preserveAspectRatio	Behaviour
`xMidYMid meet` (default)	Fit inside, centred, uniform scale
`xMidYMid slice`	Fill the viewport, crop overflow
`none`	Stretch to fill: distortion
`xMinYMin meet`	Fit, aligned top-left

Pan and zoom a map by changing the viewBox numbers.

Basic shapes



rect

```
<rect x="10" y="10" width="80"
height="50" rx="8"/>
```



circle

```
<circle cx="50" cy="50" r="40"/>
```



ellipse

```
<ellipse cx="50" cy="40" rx="45"
ry="25"/>
```



line

```
<line x1="0" y1="0" x2="90"
y2="60"/>
```



polyline

```
<polyline points="0,60 30,10 60,40
90,0"/>
```



polygon

```
<polygon points="50,0 100,90
0,90"/>
```

The path element

Command	Parameters	Draws
`M` / `m`	x y	Move to (no line)
`L` / `l`	x y	Line to
`H` / `V`	x / y	Horizontal / vertical line
`C` / `S`	x1 y1 x2 y2 x y	Cubic Bézier / smooth cubic
`Q` / `T`	x1 y1 x y	Quadratic Bézier / smooth
`A`	rx ry rot large sweep x y	Elliptical arc
`Z`	—	Close the path

```
<!-- a heart icon -->
<path d="M12 21 L4 13
         A5 5 0 0 1 12 6
         A5 5 0 0 1 20 13 Z"
      fill="#1f5fad"/>
```

Uppercase = absolute · **lowercase** = relative to the current point

In canvas: `ctx.fill(new Path2D("M12 21 L4 13 …"))`

Paint, gradients, reuse and structure

```
<svg viewBox="0 0 120 60">
  <defs>
    <linearGradient id="g" x1="0" x2="1">
      <stop offset="0" stop-color="#1f5fad"/>
      <stop offset="1" stop-color="#8db4e8"/>
    </linearGradient>
    <symbol id="star" viewBox="0 0 24 24">
      <path d="M12 2 L15 9 22 9 16 14 18 22
               12 17 6 22 8 14 2 9 9 9 Z"/>
    </symbol>
  </defs>
  <rect width="120" height="60" fill="url(#g)"/>
  <g fill="#fafaf7">
    <use href="#star" x="10" y="18" width="24"/>
    <use href="#star" x="48" y="18" width="24"/>
  </g>
</svg>
```

Element / attribute	Purpose
`fill`, `stroke`	Colour, `none`, `currentColor`, `url(#id)`
`stroke-dasharray`	Dashes; line-drawing animations
`<g>`	Group: shared style and transform
`<defs>`, `<use>`	Define once, draw many times
`<symbol>`	Icon sprites with own viewBox
`clipPath`, `mask`, `filter`	Cut, fade, blur, shadow

Four ways to put SVG in a page

Method	Example	CSS from page	JavaScript	Cached	Use for
`<img>`	`<img src="logo.svg" alt="…">`	No	No	Yes	Logos, illustrations
CSS background	`background: url(bg.svg)`	No	No	Yes	Decorative patterns
Inline `<svg>`	`<svg>…</svg>` in HTML	Yes	Yes	With the page	Icons, charts, maps
`<object>`	`<object data="map.svg">`	No	Own scripts	Yes	Legacy embeds

Security: SVG can contain scripts. Show user-uploaded SVGs only via `<img>`, never inline, unless sanitised (e.g. DOMPurify).

Styling SVG with CSS

```
.icon {
  width: 1.5em; height: 1.5em;
  fill: currentColor;            /* inherits text colour */
}
.btn:hover .icon { color: var(--accent); }

.bar { fill: #1f5fad; transition: fill .2s; }
.bar:hover { fill: #8db4e8; }

.spinner {
  transform-box: fill-box;       /* rotate around itself */
  transform-origin: center;
  animation: spin 1s linear infinite;
}
@keyframes spin { to { transform: rotate(1turn); } }
```

currentColor

Icons follow the surrounding text colour

Precedence

inline `style` > CSS rule > presentation attribute

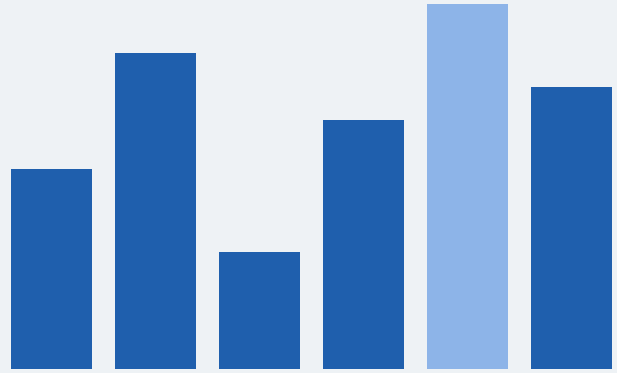
Accessibility

`role="img"` + `<title>`; `aria-hidden` if decorative

Generating SVG with JavaScript

```
const NS = "http://www.w3.org/2000/svg";
const sales = [12, 19, 7, 15, 22, 17];       // thousands €
const svg = document.querySelector("#chart"); // viewBox 0 0 300 120
const max = Math.max(...sales);

sales.forEach((v, i) => {
  const h = v / max * 100;
  const bar = document.createElementNS(NS, "rect");
  bar.setAttribute("x", 10 + i * 48);
  bar.setAttribute("y", 110 - h);
  bar.setAttribute("width", 36);
  bar.setAttribute("height", h);
  bar.setAttribute("class", "bar");
  bar.dataset.value = v;
  svg.append(bar);
});
svg.addEventListener("click", e =>
  console.log(e.target.dataset.value));       // delegation
```



Result: one `<rect>` per month

`createElementNS(NS, "rect")`, not **`createElement`**

Optimising SVG files

before

```
<!-- exported: 2.1 KB -->
<?xml version="1.0"?>
<!-- Generator: Design Tool 29.1 -->
<svg … xmlns:xlink="…" xml:space="preserve">
  <g id="Layer_1"><g>
   <path d="M12.000000,21.354000 L4.18…"/>
  </g></g>
```

after

```
<!-- SVGO: 0.4 KB -->
<svg viewBox="0 0 24 24">
  <path d="M12 21.35 4.18 13…"/>
</svg>
```

Remove

Metadata, comments, hidden layers

Round

Coordinates to 1–2 decimals

Simplify

Fewer path nodes

Compress

gzip / Brotli on the server

Canvas or SVG?

	SVG	Canvas
Model	Retained: shapes are DOM objects	Immediate: pixels only
Scaling	Always sharp	Bitmap: redraw for each size
Events	Per shape, like HTML	On the whole canvas; manual hit testing
Styling	CSS, classes, hover	Code only
Accessibility	Text, `title`, ARIA per shape	Needs a separate text alternative
Performance	Slows with many thousands of nodes	Handles huge numbers of objects
Per-pixel access	No	Yes: `getImageData`
Best for	Icons, UI, charts, maps	Photo editing, games, particles, big data

06

Business use and review

Choosing images that sell, load fast and print well

Images in business: four cases

E-commerce catalogue

AVIF/WebP photos with JPEG fallback, srcset sizes, white or transparent backgrounds, fast LCP

→ **Faster pages, better conversion**

Brand identity

Logo as SVG master; colours defined in RGB/HEX, CMYK and Pantone

→ **Consistency on screen and in print**

Dashboards and reports

SVG charts generated from data; same file on screen, in PDF and on paper

→ **Sharp, accessible, interactive**

Campaign assets

One design exported to many sizes and formats by an automated pipeline

→ **Less manual work, fewer errors**

SELF-ASSESSMENT

Review questions

1. How do additive and subtractive mixing differ? Name the complementary pairs.
2. How many colours does 24-bit RGB allow? Show the calculation.
3. Raw size of a 1024 × 768 true-colour image?
4. Convert pure red (255, 0, 0) to CMYK.
5. Why does JPEG convert to YCbCr, and what does 4:2:0 save?
6. Which JPEG step loses information, and why can it not be undone?
7. Build the Huffman code for HELLO. How many bits?
8. Choose formats for: a screenshot, a logo, a product photo.
9. What is the difference between viewBox and width/height?
10. Canvas or SVG: an interactive chart? a filter on a 12 MP photo?

RESOURCES

Bibliography and resources

Books

Li Z.-N., Drew M. S., Liu J., Fundamentals of Multimedia, 3rd ed., Springer, 2021 (ch. 3, 4, 7, 9)

Gonzalez R. C., Woods R. E., Digital Image Processing, 4th ed., Pearson, 2018

Poynton C., Digital Video and HD: Algorithms and Interfaces, 2nd ed., Morgan Kaufmann, 2012

Bellamy-Royds A., Cagle K., Storey D., Using SVG with CSS3 and HTML5, O'Reilly, 2017

Online

MDN: SVG tutorial, Canvas API tutorial, image file type guide

web.dev: Learn Images (formats, responsive images)

Squoosh: squoosh.app, compare codecs visually

SVGOMG: jakearchibald.github.io/svgomg

OKLCH picker: oklch.com

Can I use: caniuse.com (AVIF, JPEG XL)

Questions?

Next lecture: image processing in the browser — histogram, filters, convolution, workers